भारतीय प्रौद्योगिकी संस्थान जोधपुर
Indian Institute of Technology Jodhpur

MIN-HASH and LSH

Assignment-11

Machine Learning with Big Data

Submitted by

Shreyas B Gaikwad

Roll No. M25DE1042

M Tech (Data Engineering)

INTRODUCTION

The assignment is designed to understand the concepts of k-gram shingling, Jaccard similarity, Min-Hashing and Locality Sensitive Hashing (LSH) which are techniques used in large-scale data processing for efficient similarity detection. They are applied for document similarity detection, plagiarism detection, recommendation systems and information retrieval.

The first part of the assignment focuses on constructing k-grams (shingles) from textual documents. Character based and word based k-grams were generated to capture local patterns in the text. Using these shingles, the Jaccard similarity between document pairs was calculated. This step demonstrates how document similarity can be quantified using set-based representations of textual data. The results show how different shingle types influence similarity measurements between documents.

The second part of the assignment implements Min-Hashing, which is used to approximate Jaccard similarity between sets. Instead of comparing entire sets directly, multiple hash functions are used to generate compact signatures for each document. These signatures keep thde similarity relationships between documents while reducing storage and computation requirements. Experiments were conducted using different numbers of hash functions to see effect on accuracy and computational efficiency.

The third component of the assignment uses technique of LSH to identify candidate pairs of documents with high similarity. LSH divides the Min-Hash signatures into bands and uses hashing to group similar signatures together. This allows the system to avoid too many computations but still detect similar documents with high probability.

The MovieLens 100k dataset, where Min-Hashing and LSH are applied to compute similarity between users based on the movies they have rated. The exact Jaccard similarity is compared with Min-Hash approximations and the effectiveness of LSH in reducing computation while identifying similar user pairs is checked.

GITHUB LINK : **https://github.com/geek1042/mlbd_a2.git**

SOLUTIONS

1. Create k-Grams: All documents contain only lowercase letters and spaces. Duplicates were removed by storing shingles as sets.

1 (A). Min-Hash Signature using 3-grams

We estimate Jaccard similarity between D1 and D2 using Min-Hash signatures with different numbers of hash functions.

Code snippet:

```
import os
import itertools
import random
from collections import defaultdict

MINHASH_PATH = r"D:\BDML_Assign2\minhash"

def read_doc(name):
    with open(os.path.join(MINHASH_PATH, name), "r", encoding="utf-8") as f:
        return f.read().strip()

docs = {
    "D1": read_doc("D1.txt"),
    "D2": read_doc("D2.txt"),
    "D3": read_doc("D3.txt"),
    "D4": read_doc("D4.txt")
}

def char_kgrams(text, k):
    return set(text[i:i+k] for i in range(len(text)-k+1))

def word_kgrams(text, k):
    words = text.split()
    return set(tuple(words[i:i+k]) for i in range(len(words)-k+1))

def jaccard(a,b):
    return len(a & b) / len(a | b)

char2 = {k: char_kgrams(v,2) for k,v in docs.items()}
char3 = {k: char_kgrams(v,3) for k,v in docs.items()}
word2 = {k: word_kgrams(v,2) for k,v in docs.items()}

# Solution 1(A)

print("\n")
print("1(A) Min-Hash using 3-grams (D1 vs D2)")
print("\n")
```

```python
m = 20011

def generate_hash(t):
    funcs = []
    for _ in range(t):
        a = random.randint(1,m-1)
        b = random.randint(0,m-1)
        funcs.append((a,b))
    return funcs

def hash_value(x,a,b):
    return (a*x + b) % m

universe = set()
for s in char3.values():
    universe |= s
shingle_map = {g:i for i,g in enumerate(universe)}

def minhash(shingles, hash_funcs):
    indices = [shingle_map[s] for s in shingles]
    sig = []
    for a,b in hash_funcs:
        sig.append(min(hash_value(i,a,b) for i in indices))
    return sig

def approx(sig1,sig2):
    return sum(1 for a,b in zip(sig1,sig2) if a==b)/len(sig1)

exact_val = jaccard(char3["D1"], char3["D2"])
print("Exact Jaccard =", exact_val)

for t in [20,60,150,300,600]:
    funcs = generate_hash(t)
    s1 = minhash(char3["D1"], funcs)
    s2 = minhash(char3["D2"], funcs)
    print("t =",t,"Approximate Jaccard =",approx(s1,s2))
```

Output:

```
Exact Jaccard = 0.977979274611399
t = 20 Approximate Jaccard = 0.95
t = 60 Approximate Jaccard = 0.9666666666666667
t = 150 Approximate Jaccard = 0.9866666666666667
t = 300 Approximate Jaccard = 0.9766666666666667
t = 600 Approximate Jaccard = 0.98
```

As t increases, the approximation converges toward the exact Jaccard similarity. Values stabilize around t = 150 and continue to be similar till t = 600.

1(B) Construct k-grams and compute Jaccard similarity. Three types of k-grams were constructed: Character 2-grams, Character 3-grams, Word 2-grams

Code snippet:

```
def char_kgrams(text, k):
    return set(text[i:i+k] for i in range(len(text)-k+1))

def word_kgrams(text, k):
    words = text.split()
    return set(tuple(words[i:i+k]) for i in range(len(words)-k+1))

char2 = {k: char_kgrams(v,2) for k,v in docs.items()}
char3 = {k: char_kgrams(v,3) for k,v in docs.items()}
word2 = {k: word_kgrams(v,2) for k,v in docs.items()}

print("\nCharacter 2-grams")
for a,b in itertools.combinations(docs.keys(),2):
    print(a,b,jaccard(char2[a],char2[b]))

print("\nCharacter 3-grams")
for a,b in itertools.combinations(docs.keys(),2):
    print(a,b,jaccard(char3[a],char3[b]))

print("\nWord 2-grams")
for a,b in itertools.combinations(docs.keys(),2):
    print(a,b,jaccard(word2[a],word2[b]))
```

Output: Total 18 values obtained

```
1(B) Exact Jaccard for All Pairs


Character 2-grams
D1 D2 0.9811320754716981
D1 D3 0.8156996587030717
D1 D4 0.6444444444444445
D2 D3 0.8
D2 D4 0.6412698412698413
D3 D4 0.6529968454258676

Character 3-grams
D1 D2 0.977979274611399
D1 D3 0.5803571428571429
D1 D4 0.3050847457627119
D2 D3 0.5680473372781065
D2 D4 0.30590339892665475
D3 D4 0.31212381771281167

Word 2-grams
D1 D2 0.9407665505226481
D1 D3 0.18234165067178504
D1 D4 0.03024193548387097
D2 D3 0.1736641221374046
D2 D4 0.030303030303030304
D3 D4 0.01607142857142857
```

2(A) Min-Hash Approximation: Jaccard similarity between the pair of documents D1 and D2,

Code Snippet:

```
for t in [20,60,150,300,600]:
    funcs = generate_hash(t)
    s1 = minhash(char3["D1"], funcs)
    s2 = minhash(char3["D2"], funcs)
    print("t =",t,"Approximate Jaccard =",approx(s1,s2))
```

Output:

```
2(A) Min-Hash Approximation


t = 20 Approximate Jaccard = 1.0
t = 60 Approximate Jaccard = 0.9333333333333333
t = 150 Approximate Jaccard = 0.98
t = 300 Approximate Jaccard = 0.98
t = 600 Approximate Jaccard = 0.9733333333333334
```

2(B) To find a good value of t, we modify the code above to get more values in detail.

Code Snippet:

```
for t in
[10,20,30,40,50,60,70,80,90,100,110,120,130,140,150,160,170,180,190,200,210,220,230,240,250,26
0,270,280,290,300,450,600]:
    funcs = generate_hash(t)
    s1 = minhash(char3["D1"], funcs)
    s2 = minhash(char3["D2"], funcs)
    print("t =",t,"Approximate Jaccard =",approx(s1,s2))
```

Output: As given below, the values below 30 show variations and fluctuate more than values from 40. The values seem to stabilize around 0.98 at 160 onwards with less fluctuation. There is no major improvement even at higher values, hence 160-180 can be considered as better range for optimal calculations.

```
2(B) Good value of t
t = 10 Approximate Jaccard = 1.0
t = 20 Approximate Jaccard = 1.0
t = 30 Approximate Jaccard = 1.0
t = 40 Approximate Jaccard = 0.975
t = 50 Approximate Jaccard = 0.98
t = 60 Approximate Jaccard = 0.9666666666666667
t = 70 Approximate Jaccard = 1.0
t = 80 Approximate Jaccard = 0.9875
t = 90 Approximate Jaccard = 0.9444444444444444
t = 100 Approximate Jaccard = 0.98
t = 110 Approximate Jaccard = 0.990909090909091
t = 120 Approximate Jaccard = 0.9916666666666667
t = 130 Approximate Jaccard = 0.9923076923076923
t = 140 Approximate Jaccard = 0.9714285714285714
t = 150 Approximate Jaccard = 0.98
t = 160 Approximate Jaccard = 0.98125
t = 170 Approximate Jaccard = 0.9823529411764705
t = 180 Approximate Jaccard = 0.9944444444444445
t = 190 Approximate Jaccard = 0.9842105263157894
t = 200 Approximate Jaccard = 0.975
t = 210 Approximate Jaccard = 0.9809523809523809
t = 220 Approximate Jaccard = 0.9727272727272728
t = 230 Approximate Jaccard = 0.9695652173913043
t = 240 Approximate Jaccard = 0.9666666666666667
t = 250 Approximate Jaccard = 0.984
t = 260 Approximate Jaccard = 0.9846153846153847
t = 270 Approximate Jaccard = 0.9740740740740741
t = 280 Approximate Jaccard = 0.9857142857142858
t = 290 Approximate Jaccard = 0.9827586206896551
t = 300 Approximate Jaccard = 0.9866666666666667
t = 450 Approximate Jaccard = 0.98
t = 600 Approximate Jaccard = 0.9866666666666667
```

3(A) Probability of each pair of the four documents (using 3-grams) being estimated to have a similarity greater than (τ).

Solution:

We use a MinHash signature of length t=160, which was found best suitable in the above. The signature matrix is divided into bands (b) and rows per band (r) such that:

$$t=r\times b$$

The aim is to choose values of r and b so that the S-curve sharply separates document pairs above and below the similarity threshold:

$$\tau=0.7$$

We select 40 and 4 value, as then the total 4 X 40 becomes 160 which is min value we got in the above.
Probability at Threshold
s=0.7

Using the formula: $f(s)=1-(1-s^r)^b$

Substituting the values: $f(0.7) = 1-(1-0.74)^{40}$
$f(0.7) = 1-(1-0.2401)^{40}$
$f(0.7) = 1-(0.7599)^{40}$
$f(0.7) \approx 0.99998$

3(B) Probability of each pair of the four documents (using 3-grams) being estimated to have a similarity greater than (τ)

Code Snippet:

```
def lsh_prob(s,r,b):
    return 1 - (1 - s**r)**b

for a,b_doc in itertools.combinations(docs.keys(),2):
    sim = jaccard(char3[a],char3[b_doc])
    print(a,b_doc,"Probability =",lsh_prob(sim,r,b))
```

Output:

```
3(B) LSH Probability for Each Pair


D1 D2 Probability = 1.0
D1 D3 Probability = 0.9919044206534928
D1 D4 Probability = 0.29392984078239415
D2 D3 Probability = 0.9876980128156715
D2 D4 Probability = 0.2965848032379953
D3 D4 Probability = 0.3171289518403788
```

- D1-D2, D1-D3 and D2-D3 have probabilities close to 1, indicating strong similarity which is also high than Jaccard Similarity values.
- D1-D4, D2-D4 and D3-D4 have probabilities around 0.3,which indicates very less similarily.

4(A). Min-Hashing on MovieLens dataset

Code Snippet:

```
MOVIELENS_PATH = r"D:\BDML_Assign2\u.data"

def load_movielens():
    users = defaultdict(set)
    with open(MOVIELENS_PATH,"r") as f:
        for line in f:
            user,movie,_,_ = line.strip().split("\t")
            users[int(user)].add(int(movie))
    return users

users = load_movielens()

keys = list(users.keys())
exact_pairs = set()

for i in range(len(keys)):
    for j in range(i+1,len(keys)):
        if jaccard(users[keys[i]],users[keys[j]]) >= 0.5:
            exact_pairs.add((keys[i],keys[j]))

print("Exact pairs count =",len(exact_pairs))
```

Output:

```
4(A) Exact Jaccard >= 0.5 (MovieLens)

Exact pairs count = 10
```

4(B). Output pairs that have an estimated similarity of at least 0.5 and report the number of false positives and false negatives that is obtained.

Code Snippet:

```
def user_minhash(users,t):
    universe = set()
    for s in users.values():
        universe |= s
    mapping = {g:i for i,g in enumerate(universe)}
    funcs = generate_hash(t)
    signatures = {}
    for u in users:
        indices = [mapping[i] for i in users[u]]
        sig = []
        for a,b in funcs:
            sig.append(min(hash_value(i,a,b) for i in indices))
        signatures[u] = sig
```

```
    return signatures

def approx_user_pairs(signatures,threshold):
    keys = list(signatures.keys())
    pairs = set()
    for i in range(len(keys)):
        for j in range(i+1,len(keys)):
            if approx(signatures[keys[i]],signatures[keys[j]]) >= threshold:
                pairs.add((keys[i],keys[j]))
    return pairs

for t in [50,100,200]:
    fp_total = 0
    fn_total = 0
    for _ in range(5):
        sigs = user_minhash(users,t)
        approx_pairs = approx_user_pairs(sigs,0.5)
        fp_total += len(approx_pairs - exact_pairs)
        fn_total += len(exact_pairs - approx_pairs)
    print("t =",t,"avg FP =",fp_total/5,"avg FN =",fn_total/5)
```

Output:

```
4(B) Min-Hash Results (Average over 5 runs)


t = 50 avg FP = 130.8 avg FN = 3.0
t = 100 avg FP = 27.2 avg FN = 1.8
t = 200 avg FP = 10.0 avg FN = 1.0
```

Hence it is seen that increasing the Hash functions reduces the false values to a great extent.

5. LSH on MovieLens dataset: Find candidate pairs with a similarity of at least 0.6. Report the number of false positives and false negatives, taking the average over 5 runs.

Code Snippet:

```
def lsh_users(signatures,r,b):
    bands = []
    for i in range(b):
        bands.append(defaultdict(list))
    for u,sig in signatures.items():
        for i in range(b):
            start = i*r
            end = start+r
            key = tuple(sig[start:end])
```

```
            bands[i][key].append(u)
    candidates = set()
    for band in bands:
        for bucket in band.values():
            if len(bucket)>1:
                for pair in itertools.combinations(bucket,2):
                    candidates.add(tuple(sorted(pair)))
    return candidates

configs = [
    (50,5,10),
    (100,5,20),
    (200,5,40),
    (200,10,20)
]

for t,r,b in configs:
    fp_total = 0
    fn_total = 0
    for _ in range(5):
        sigs = user_minhash(users,t)
        candidates = lsh_users(sigs,r,b)
        predicted = set()
        for u,v in candidates:
            if approx(sigs[u],sigs[v]) >= 0.6:
                predicted.add((u,v))
        fp_total += len(predicted - exact_pairs)
        fn_total += len(exact_pairs - predicted)
    print("t =",t,"r =",r,"b =",b,"avg FP =",fp_total/5,"avg FN =",fn_total/5)
```

Output:

```
5) LSH on MovieLens (Average over 5 runs)


t = 50 r = 5 b = 10 avg FP = 2.2 avg FN = 6.4
t = 100 r = 5 b = 20 avg FP = 0.6 avg FN = 7.2
t = 200 r = 5 b = 40 avg FP = 0.0 avg FN = 6.8
t = 200 r = 10 b = 20 avg FP = 0.0 avg FN = 8.2
```

With increase in number of rows, the false positives decrease.
And with increase in number of bands, false negative increases.
We see that for rows=5 and rows per band=40, false positives is Nil and false negatives are almost minimum.

Complete Code:

```
import os
import itertools
import random
from collections import defaultdict

MINHASH_PATH = r"D:\BDML_Assign2\minhash"
MOVIELENS_PATH = r"D:\BDML_Assign2\u.data"

def read_doc(name):
    with open(os.path.join(MINHASH_PATH, name), "r", encoding="utf-8") as f:
        return f.read().strip()

docs = {
    "D1": read_doc("D1.txt"),
    "D2": read_doc("D2.txt"),
    "D3": read_doc("D3.txt"),
    "D4": read_doc("D4.txt")
}

def char_kgrams(text, k):
    return set(text[i:i+k] for i in range(len(text)-k+1))

def word_kgrams(text, k):
    words = text.split()
    return set(tuple(words[i:i+k]) for i in range(len(words)-k+1))

def jaccard(a,b):
    return len(a & b) / len(a | b)

char2 = {k: char_kgrams(v,2) for k,v in docs.items()}
char3 = {k: char_kgrams(v,3) for k,v in docs.items()}
word2 = {k: word_kgrams(v,2) for k,v in docs.items()}

# Solution 1(A)

print("\n")
print("1(A) Min-Hash using 3-grams (D1 vs D2)")
print("\n")

m = 20011

def generate_hash(t):
    funcs = []
    for _ in range(t):
        a = random.randint(1,m-1)
        b = random.randint(0,m-1)
        funcs.append((a,b))
    return funcs

def hash_value(x,a,b):
    return (a*x + b) % m
```

```python
universe = set()
for s in char3.values():
    universe |= s
shingle_map = {g:i for i,g in enumerate(universe)}

def minhash(shingles, hash_funcs):
    indices = [shingle_map[s] for s in shingles]
    sig = []
    for a,b in hash_funcs:
        sig.append(min(hash_value(i,a,b) for i in indices))
    return sig

def approx(sig1,sig2):
    return sum(1 for a,b in zip(sig1,sig2) if a==b)/len(sig1)

exact_val = jaccard(char3["D1"], char3["D2"])
print("Exact Jaccard =", exact_val)

for t in [20,60,150,300,600]:
    funcs = generate_hash(t)
    s1 = minhash(char3["D1"], funcs)
    s2 = minhash(char3["D2"], funcs)
    print("t =",t,"Approximate Jaccard =",approx(s1,s2))


# Solution 1(B)

print("\n")
print("1(B) Exact Jaccard for All Pairs")
print("\n")

print("\nCharacter 2-grams")
for a,b in itertools.combinations(docs.keys(),2):
    print(a,b,jaccard(char2[a],char2[b]))

print("\nCharacter 3-grams")
for a,b in itertools.combinations(docs.keys(),2):
    print(a,b,jaccard(char3[a],char3[b]))

print("\nWord 2-grams")
for a,b in itertools.combinations(docs.keys(),2):
    print(a,b,jaccard(word2[a],word2[b]))

# Solution 2(A)

print("\n")
print("2(A) Min-Hash Approximation")
print("\n")

for t in [20,60,150,300,600]:
    funcs = generate_hash(t)
    s1 = minhash(char3["D1"], funcs)
    s2 = minhash(char3["D2"], funcs)
    print("t =",t,"Approximate Jaccard =",approx(s1,s2))
```

```python
# Solution 2(B)

print("\n")
print("2(B) Good value of t")

for t in
[10,20,30,40,50,60,70,80,90,100,110,120,130,140,150,160,170,180,190,200,210,220,230,240,250,26
0,270,280,290,300,450,600]:
    funcs = generate_hash(t)
    s1 = minhash(char3["D1"], funcs)
    s2 = minhash(char3["D2"], funcs)
    print("t =",t,"Approximate Jaccard =",approx(s1,s2))

print("\n")

# Solution 3(B)

print("\n")
print("3(B) LSH Probability for Each Pair")
print("\n")
r=4
b=40

def lsh_prob(s,r,b):
    return 1 - (1 - s**r)**b

for a,b_doc in itertools.combinations(docs.keys(),2):
    sim = jaccard(char3[a],char3[b_doc])
    print(a,b_doc,"Probability =",lsh_prob(sim,r,b))

# Solution 4(A)

print("\n")
print("4(A) Exact Jaccard >= 0.5 (MovieLens)")
print("\n")

def load_movielens():
    users = defaultdict(set)
    with open(MOVIELENS_PATH,"r") as f:
        for line in f:
            user,movie,_,_ = line.strip().split("\t")
            users[int(user)].add(int(movie))
    return users

users = load_movielens()

keys = list(users.keys())
exact_pairs = set()

for i in range(len(keys)):
    for j in range(i+1,len(keys)):
        if jaccard(users[keys[i]],users[keys[j]]) >= 0.5:
            exact_pairs.add((keys[i],keys[j]))
```

```python
print("Exact pairs count =",len(exact_pairs))

# Solution 4(B)

print("\n")
print("4(B) Min-Hash Results (Average over 5 runs)")
print("\n")

def user_minhash(users,t):
    universe = set()
    for s in users.values():
        universe |= s
    mapping = {g:i for i,g in enumerate(universe)}
    funcs = generate_hash(t)
    signatures = {}
    for u in users:
        indices = [mapping[i] for i in users[u]]
        sig = []
        for a,b in funcs:
            sig.append(min(hash_value(i,a,b) for i in indices))
        signatures[u] = sig
    return signatures

def approx_user_pairs(signatures,threshold):
    keys = list(signatures.keys())
    pairs = set()
    for i in range(len(keys)):
        for j in range(i+1,len(keys)):
            if approx(signatures[keys[i]],signatures[keys[j]]) >= threshold:
                pairs.add((keys[i],keys[j]))
    return pairs

for t in [50,100,200]:
    fp_total = 0
    fn_total = 0
    for _ in range(5):
        sigs = user_minhash(users,t)
        approx_pairs = approx_user_pairs(sigs,0.5)
        fp_total += len(approx_pairs - exact_pairs)
        fn_total += len(exact_pairs - approx_pairs)
    print("t =",t,"avg FP =",fp_total/5,"avg FN =",fn_total/5)

# Solution 5

print("\n")
print("5) LSH on MovieLens (Average over 5 runs)")
print("\n")

def lsh_users(signatures,r,b):
    bands = []
    for i in range(b):
        bands.append(defaultdict(list))
    for u,sig in signatures.items():
```

```
        for i in range(b):
            start = i*r
            end = start+r
            key = tuple(sig[start:end])
            bands[i][key].append(u)
    candidates = set()
    for band in bands:
        for bucket in band.values():
            if len(bucket)>1:
                for pair in itertools.combinations(bucket,2):
                    candidates.add(tuple(sorted(pair)))
    return candidates

configs = [
    (50,5,10),
    (100,5,20),
    (200,5,40),
    (200,10,20)
]

for t,r,b in configs:
    fp_total = 0
    fn_total = 0
    for _ in range(5):
        sigs = user_minhash(users,t)
        candidates = lsh_users(sigs,r,b)
        predicted = set()
        for u,v in candidates:
            if approx(sigs[u],sigs[v]) >= 0.6:
                predicted.add((u,v))
        fp_total += len(predicted - exact_pairs)
        fn_total += len(exact_pairs - predicted)
    print("t =",t,"r =",r,"b =",b,"avg FP =",fp_total/5,"avg FN =",fn_total/5)
```

Terminal Output:

```
D:\BDML_Assign2> python assign.py

1(A) Min-Hash using 3-grams (D1 vs D2)

Exact Jaccard = 0.977979274611399
t = 20 Approximate Jaccard = 1.0
t = 60 Approximate Jaccard = 0.9833333333333333
t = 150 Approximate Jaccard = 0.9933333333333333
t = 300 Approximate Jaccard = 0.9866666666666667
t = 600 Approximate Jaccard = 0.9716666666666667

1(B) Exact Jaccard for All Pairs

Character 2-grams
D1 D2 0.9811320754716981
D1 D3 0.8156996587030717
D1 D4 0.6444444444444445
```

```
D2 D3 0.8
D2 D4 0.6412698412698413
D3 D4 0.6529968454258676

Character 3-grams
D1 D2 0.977979274611399
D1 D3 0.5803571428571429
D1 D4 0.3050847457627119
D2 D3 0.5680473372781065
D2 D4 0.30590339892665475
D3 D4 0.31212381771281167

Word 2-grams
D1 D2 0.9407665505226481
D1 D3 0.18234165067178504
D1 D4 0.03024193548387097
D2 D3 0.1736641221374046
D2 D4 0.030303030303030304
D3 D4 0.01607142857142857


2(A) Min-Hash Approximation

t = 20 Approximate Jaccard = 1.0
t = 60 Approximate Jaccard = 0.9333333333333333
t = 150 Approximate Jaccard = 0.9733333333333334
t = 300 Approximate Jaccard = 0.9666666666666667
t = 600 Approximate Jaccard = 0.98


2(B) Good value of t
t = 10 Approximate Jaccard = 1.0
t = 20 Approximate Jaccard = 0.95
t = 30 Approximate Jaccard = 1.0
t = 40 Approximate Jaccard = 0.975
t = 50 Approximate Jaccard = 0.98
t = 60 Approximate Jaccard = 0.9833333333333333
t = 70 Approximate Jaccard = 0.9714285714285714
t = 80 Approximate Jaccard = 0.975
t = 90 Approximate Jaccard = 0.9888888888888889
t = 100 Approximate Jaccard = 0.98
t = 110 Approximate Jaccard = 0.990909090909091
t = 120 Approximate Jaccard = 0.9833333333333333
t = 130 Approximate Jaccard = 0.9692307692307692
t = 140 Approximate Jaccard = 0.9857142857142858
t = 150 Approximate Jaccard = 0.98
t = 160 Approximate Jaccard = 0.9875
t = 170 Approximate Jaccard = 0.9823529411764705
t = 180 Approximate Jaccard = 0.9888888888888889
t = 190 Approximate Jaccard = 0.9789473684210527
t = 200 Approximate Jaccard = 0.965
t = 210 Approximate Jaccard = 0.9904761904761905
t = 220 Approximate Jaccard = 0.9681818181818181
t = 230 Approximate Jaccard = 0.9869565217391304
```

```
t = 240 Approximate Jaccard = 0.9708333333333333
t = 250 Approximate Jaccard = 0.996
t = 260 Approximate Jaccard = 0.9884615384615385
t = 270 Approximate Jaccard = 0.9777777777777777
t = 280 Approximate Jaccard = 0.9642857142857143
t = 290 Approximate Jaccard = 0.9827586206896551
t = 300 Approximate Jaccard = 0.9766666666666667
t = 450 Approximate Jaccard = 0.9822222222222222
t = 600 Approximate Jaccard = 0.9666666666666667


3(B) LSH Probability for Each Pair

D1 D2 Probability = 1.0
D1 D3 Probability = 0.9919044206534928
D1 D4 Probability = 0.29392984078239415
D2 D3 Probability = 0.9876980128156715
D2 D4 Probability = 0.2965848032379953
D3 D4 Probability = 0.3171289518403788


4(A) Exact Jaccard >= 0.5 (MovieLens)

Exact pairs count = 10


4(B) Min-Hash Results (Average over 5 runs)

t = 50 avg FP = 172.0 avg FN = 2.6
t = 100 avg FP = 39.0 avg FN = 2.4
t = 200 avg FP = 6.8 avg FN = 2.4


5) LSH on MovieLens (Average over 5 runs)

t = 50 r = 5 b = 10 avg FP = 2.2 avg FN = 6.4
t = 100 r = 5 b = 20 avg FP = 0.6 avg FN = 7.2
t = 200 r = 5 b = 40 avg FP = 0.0 avg FN = 6.8
t = 200 r = 10 b = 20 avg FP = 0.0 avg FN = 8.2

D:\BDML_Assign2>
```